

Auditoria técnica do Clipper — raio-X ponta a ponta

Leitura apenas. Nenhum código alterado, nenhum commit, nenhuma correção. Gerado em 2026-09-19 13:36 UTC a partir do código atual do repositório (HEAD d952976). Relatórios anteriores tratados como evidência, não como verdade — cada afirmação abaixo foi conferida no código citado.

1. Raio-X por etapa (o fluxo REAL)

1.1 Extração de áudio — `core/backends.py`: `audio_chunks()`

- Entra: caminho do vídeo + offset inicial (0, 30, 60, ...).
- Sai: arquivos WAV temporários (1 por vez), 16 kHz mono PCM + offset em segundos.
- Comando: `ffmpeg -ss <off> -t 30 -i video -ar 16000 -ac 1 pcm_s16le`.
- Timestamps: não existem aqui (áudio puro). Premissa: `-ss` antes do `-i`

(seek rápido) é amostra-exato o suficiente para fala; erro, se houver, seria de milissegundos no corte do chunk, nunca deslocamento de legenda.

- Nada é descartado; nada artificial é criado. Opcional: `-af <filtro>` só com `CLIPPER_TRANSCRIBE_AUDIO_FILTER` (off por padrão).

1.2 Chunks — `core/backends.py`: `CHUNK_SECONDS = 30`

- Por que 30 s: janela nativa do Whisper; cada chunk = ~1 MB, 1 processo `whisper-cli` sequencial (timeout 300 s), removido após uso.
- `chunk_offset` = o `-ss` da extração. O `whisper-cli` sempre responde como se o áudio começasse em 0 — os offsets do JSON são locais ao chunk.

1.3 Whisper — `core/backends.py`: `_whisper_cmd()` + `transcribe_vulkan()`

- Comando atual (verificado linha 159-160): ``whisper-cli -m <ggml> -f <wav> -ojf -of <stem> -l <lang|auto> -nt``. Nenhuma flag de VAD.
- Backend: `whisper.cpp` build Vulkan (B580). Fallback: OpenVINO GPU, depois `faster-whisper` CPU int8 com `vad_filter=False` (`core/transcribe.py`:49).
- Sai por chunk: JSON ``transcription[{{text, offsets{from,to}ms, tokens[{{text, offsets{from,to}ms, p}}]}``.
- Premissa: modelo `ggml-medium.bin` em `models/`; sem ele, erro claro (não silencioso).

1.4 VAD — estado atual: AUSENTE do caminho padrão

- O binário do modelo Silero pode existir no disco, mas nada o carrega: `_whisper_cmd()` não tem `-vm/--vad` (teste `test_backends.py` trava isso), CPU usa `vad_filter=False`, e `core/translab.py` nunca teve VAD.
- Efeito prático: o Whisper ouve o áudio corrido do chunk, na timeline única do chunk. Sem compressão de timeline, sem remapeamento (ver §4).

1.5 JSON → Word — parse em `transcribe_vulkan()` (linhas 194-214)

- Para cada token com texto não-vazio e não-especial (`[eot]`, `[sot]`...):
`Word(text, offset + from/1000, offset + from/1000→to/1000)`.
- Unidade: ms → s (divisão por 1000, float). Referência: segundos absolutos

do vídeo original.

- Descartado aqui: tokens de controle e strings vazias (nunca foram fala).
- Criado aqui (único tempo fabricado do sistema): `_even_words()` distribui palavras uniformemente só quando o segmento veio sem tokens válidos.
- Segmentos guardam `Segment(text limpo, start, end, words)` com a mesma fórmula aplicada aos offsets do segmento.

1.6 Transcript global — `core/transcribe.py: transcribe()`

- Concatenação ordenada dos segmentos de todos os chunks. Sem merge, sem dedup, sem retoque, sem reordenação além da ordem dos chunks.
- Cache opcional (`.cache/<sha1>.transcript.json, fingerprint = caminho+tamanho+mtime+modelo+idioma`): serializa e reconstrói floats sem arredondamento intencional. Risco conhecido: trocar código/modelo sem trocar o vídeo reaproveita transcript velho — em investigação, usar `--force-retranscribe`.

1.7 Scoring — `core/scoring.py`

- Entra: texto dos candidatos (+ duração/energia/speech_rate). Sai: nota, título, hashtags (LLM remoto, só texto).
- Timestamps: não entram, não saem, não são tocados. Premissa: o prompt exige título-recorte fiel ao texto (regra anti-alucinação).

1.8 Seleção do clip — `core/candidates.py: build/snap_all + core/selection.py`

- `build()`: janelas deslizantes sobre bounds de segmento, duração em `[--min-duration, --max-duration]` (default 20–90 s), passo 20 s.
- `_snap_one()`: cada borda gruda na fronteira de palavra mais próxima (± 1.5 s), ganha `--pad` (0.8 s) e é clampada: nunca além do MAX, nunca antes de 0, nunca além do fim da mídia; abaixo do MIN, expande contexto.
- `select()`: filtra por nota, NMS com decaimento + diversidade por 10 min. Não move bordas.
- Saem `clip_start/clip_end` absolutos. Transformação temporal aqui: move BORDAS do clip (sempre em direção a timestamps reais de word); jamais move o instante de uma palavra.

1.9 Seleção das words do clip — `core/video.py: _group_cue_words()`

- Regra: word pertence ao clip se `w.end > clip_start` and `w.start < clip_end`.
- Words degeneradas (`end <= start`) descartadas aqui (ponto único).
- Premissa: pertinência é por overlap, não por contenção total (palavra a cavalo da borda participa).

1.10 Grouping — `core/video.py: _group_cues() + _split_lines()`

- `_group_cues()`: agrupa por gap > 0.4 s, pontuação . ! ?, 8 words ou 48 chars. Cue = (start da primeira word, end da última word com piso visual de 1.0 s clampado contra a próxima cue, texto limpo em maiúsculas). Tudo ainda absoluto.
- `_split_lines()`: única conversão abs → rel do sistema (`rel = abs - clip_start, clamp [0, duration]`), quebra linhas (≤ 24 chars, ≤ 2 por bloco, divide duração se >1 bloco).

- Criado artificialmente: só a *duração visual mínima* (e só a parte que excede o span real é clampada). Início de cue é sempre start de word real.

1.11 Linhas de legenda — SRT / ASS

- `build_srt()`: numera e formata HH:MM:SS,mmm. `build_ass()`: mesmo conjunto + estilos (Montserrat ExtraBold, PlayRes = frame real, pop 280 ms, hook do título, lanes por MarginV, destaque de palavras do título). Invariante testada: SRT == ASS (mesma timeline).
- Micro-comportamento verificado: `_srt_time` trunca (não arredonda) milissegundos — erro máximo sub-1 ms, irrelevante para fala.

1.12 Vídeo final — `core/video.py: cut()` + FFmpeg

- Corte 0-based: `trim/atrim + setpts/asespts`, legenda queimada via filtro `subtitles`. Medido ponta a ponta: `-ss` de saída deslocaria a legenda (o filtro avalia na timeline do demux); `trim+setpts` alinha vídeo, áudio e legenda no zero. Encode `h264_qsv` (fallback `libx264`).
- O FFmpeg não desloca timeline: ele casa o zero do corte com o zero do SRT.

2. Trajetória de um timestamp (palavra "agora", 55.20 → 55.48)

Cenário: chunk 30–60 (`chunk_offset = 30`), clip escolhido [`50.0, 80.0`] (`clip_start = 50.0`).

Etapa	start	end	unidade	referência	transformação
--- --- --- --- ---					
Whisper/token	25.20	25.48	s (de ms)	chunk (0 = início do chunk)	medição original (atenção do modelo)
JSON	25200	25480	ms inteiros	chunk	serialização (×1000, exata)
Word (parse)	55.20	55.48	s float	vídeo original	$30 + 25200/1000, 30 + 25480/1000$ (soma exata)
transcript/cache	55.20	55.48	s float	vídeo original	cópia (round-trip JSON sem retoque)
clip selection	55.20	55.48	s float	vídeo original	nenhuma na word; pertinência: $55.48 > 50.0$ and $55.20 < 80.0$ □
cue (<code>_group_cues</code>)	55.20	≥55.48*	s float	vídeo original	início copiado da word; *fim = end da última word do grupo (+piso 1 s)
relativo (<code>_split_lines</code>)	5.20	5.48	s float	corte (0 = <code>clip_start</code>)	$55.20 - 50.0, 55.48 - 50.0$ (única conversão)
SRT	00:00:05,200	00:00:05,480	ms truncados	corte	formatação de texto
ASS	0:00:05.20	0:00:05.48	centissegundos	corte	formatação de texto
FFmpeg	5.20	5.48	s na timeline do clip	corte 0-based	<code>trim+setpts</code> casa os zeros; queima

Verificação no código de cada linha: §1.5 (parse), §1.9 (pertinência), §1.10 (cue/rel), §1.11 (formatos), §1.12 (corte). Nenhuma etapa entre a medição e a tela soma, multiplica ou interpola o instante da palavra.

3. Três coisas diferentes (não confundir)

A. Erro do Whisper (origem)

- Não produziu a palavra (silêncio no transcript onde há fala no áudio).
- Produziu com instante errado (ex: jitter de ~0.5 s; span absurdo tipo "é" 0.0→17.2 visto em work/videofull).
- Produziu sem tempo mensurável (47.80→47.80 — degenerado, descartado corretamente; o texto existe, o tempo não).
- Assinatura: erro varia por trecho, sem padrão de fase; cada word erra por si.

B. Erro de pipeline (não encontrado no código atual)

- Seria: reescrever `Word.start/end` após o parse. Percorridas todas as etapas (§1): não existe nenhum caminho que reescreva `start/end` de `Word`. Candidatos checados e inocentados: `cache` (cópia), `snap` (move bordas, não words), `grouping` (início sempre = word real), `_split_lines` (subtração exata), `SRT/ASS` (formatação).
- O único tempo fabricado é `_even_words` (segmento sem tokens) — honesto e documentado, não erro.

C. Erro de render/exibição

- Seria: legenda certa no SRT mas queimada deslocada. Inocentado: `trim+setpts` 0-based medido; `SRT == ASS` testado; lanes são espaciais (`MarginV`), nunca temporais.

Caso histórico que misturava A e B (era VAD, hoje removido)

Com VAD, segmentos do JSON vinham remapeados para a timeline original (`whisper_full_get_segment_t0/t1` → `map_processed_to_original_time`) enquanto tokens vinham crus na timeline comprimida (`whisper_full_get_token_data` retorna `result_all` sem mapeamento — `src/whisper.cpp:8221`). Como janelas usam segmentos e captions usam words, texto certo caía no momento errado com deslocamento variável por trecho. Removido do caminho padrão por evidência.

4. Revalidação das conclusões anteriores (amostragem do código atual)

Conclusão	Veredito nesta auditoria
--- ---	
degenerados descartados em ponto único	CONFIRMADO (<code>_group_cues</code> , teste 21163e9 presente no git)
clamp da extensão de 1 s	CONFIRMADO (código + d3f844d no git)
sem VAD no padrão	CONFIRMADO (<code>_whisper_cmd</code> sem flags; <code>vad_filter=False</code> ; teste trava)
chunk offset correto	CONFIRMADO (soma única no parse; sem outro caminho)
abs → rel único e correto	CONFIRMADO (<code>_split_lines</code> somente)
SRT == ASS	CONFIRMADO (mesmo núcleo; testes passam)
FFmpeg/PTS	CONFIRMADO (<code>trim+setpts</code> ; sem -ss de saída)
range de clips respeitado	CONFIRMADO (<code>build + snap + validação 0 < min <= max <= 600</code>)
filtros de áudio sem ganho objetivo	PROVÁVEL (matriz medida; amostra pequena, 1 vídeo clipado)

degenerados descartados em ponto único	CONFIRMADO (<code>_group_cues</code> , teste 21163e9 presente no git)
jitter residual ~0.5 s é do modelo	PROVÁVEL (nada no pipeline o explica; falta medição dedicada sem VAD)
"55 s ouvido" do caso videofull	DESCONHECIDO (dado posiciona fala em 30–47.8; audição marcada pendente)

Suíte atual: 128 testes, todos passando (conferido nesta auditoria).

5. O que olhar primeiro (sem mexer em nada)

1. Audição marcada de `videofull.mkv` 25–60 s: o início real da frase "Agora eu vou matar..." decide se o sistema está consistente (~30 s) ou se o modelo errou a medição (~55 s). 2. Retranscrever o chunk 30–60 no pipeline atual (sem VAD): o dado existente é da era VAD e pode já estar superado. 3. Só com divergência reproduzida no pipeline atual, abrir `caption-debug.txt` (que já distingue `end <= start` de `outside_selected_clip`) e procurar a primeira divergência — nunca `_group_cues()` antes disso.